Lab Manual

Large Language Models and Query Engineering (0135444)

2026

Prepared By: Dr. Akram Fatayer

Revised By:

Al-Zaytoonah Private University of Jordan

Faculty of Science and Information Technology

Lab Manual

Course Name: Large Language Models and Query Engineering

Course Number: 0135444

Department Name: Data Science and Artificial Intelligence

Prepared by: Dr. Akram Fatayer

Table of Contents

Number	Subject	Page
1	Lab Manual Overview	3
2	Required Tools and Resources	4
3	Rules and Policies	4
4	Learning Outcomes	5
5	Exercise 1: Hello, LLM	7
6	Exercise 2: Tokens and Embeddings	9
7	Exercise 3: Attention from Scratch	11
8	Exercise 4: Decoding	13
9	Exercise 5: Adapting a Model	14
10	Exercise 6: A Prompt Test Harness	15

Lab Manual Overview

Every week of this course pairs a lecture with a 75-minute lab session in the department's GPU lab. In the lab, students run open-weight language models themselves, write the key algorithms in PyTorch, and measure how the models behave in English and in Arabic. All lab notebooks are in the course repository (github.com/akramfatayer/zuj-llm) and also open in Google Colab for work outside class.

IT LAB Manual (Large Language Models and Query Engineering)

The six experiments in this manual (Exercises 1–6, weeks 1–6) are the graded laboratory experiments of the course. Each one is explained by the instructor at the start of the session, carried out by students in pairs, and assessed by an assignment linked to that experiment, worth 5 marks (30 marks in total). The labs of weeks 7–13 (prompt chains, structured outputs, semantic search, RAG, tool-calling agents, evaluation and red-teaming) are formative practice: they are not graded separately, and students reuse them in the team project.

Course-specific learning goals of the lab:

- run, inspect and question an open-weight LLM rather than treat it as a black box;
- implement the core mechanisms (the generation loop, tokenization measurements, attention, decoding strategies, LoRA adaptation) and check them against reference implementations;
- measure, not guess: every experiment ends with numbers, plots or a log that support a written conclusion;
- compare English, Modern Standard Arabic and Jordanian dialect throughout, and explain the differences from how the models work.

Required Tools and Resources

Number	Name
1	**GPU lab PC (one per pair)**: Python 3.11, PyTorch with CUDA, Hugging Face Transformers, JupyterLab; the lab models are downloaded in advance.
2	**Google Colab**: Free T4 GPU runtime for finishing experiments outside class; every notebook has an Open in Colab link.
3	**Open-weight models**: Qwen2.5-1.5B-Instruct (default generator), GPT-2, mBERT, XLM-R, AraBERT and a multilingual MiniLM embedding model, all with open licences.
4	**Course repository**: github.com/akramfatayer/zuj-llm: lab notebooks, lecture slides and assignment briefs.
5	**Course page and interactive modules**: akramfatayer.github.io/courses/llm: browser-based modules that prepare each lab.
6	**Moodle**: Submission of every graded experiment and feedback on it.

Rules and Policies

❖ Before you come to the lab session

- You can bring your notebook so that you can keep your work with you
- No food or beverage is allowed in the lab

- Be sure to come on time to make sure you follow lab exercises with other students.
- Be sure to remember your student account username and password

❖ While being in the lab session

- Interact with the lab supervisor. Do not hesitate to ask him questions
- Maintain silence and discipline while being in the lab
- Show cooperation with your fellow students
- Log in to your computer using your student account
- Make sure you complete your lab exercises

❖ After finishing the lab session

- Make sure the place you are using is clean
- Put your chair or any material you have used back to its proper place
- Make sure to logout of your student account

❖ Attendance

- You are required to come on time, otherwise you will be considered absent
- Absence from lab sessions is recorded and will be used for absence warning system

❖ Grading

- Each of the six graded experiments (Exercises 1–6) is worth 5 marks and is assessed by an assignment linked to that experiment: 30 marks in total.
- Submit each graded experiment on Moodle by the start of the next lab session. Late work loses 10% per day for up to three days.
- Notebook submissions are checked automatically for completeness and correctness; the instructor reads the written answers and may adjust the mark. Marks are recorded on the Lab Experiment Evaluation Form.

Learning Outcomes

Exercise Number	Learning Outcome
Exercise 1	Run an open-weight LLM, read its next-token probabilities, write the generation loop and explain six typical failures (K1).
Exercise 2	Measure tokenizer fertility and token cost in English, MSA and dialect, and compare static and contextual embeddings (K1).

Exercise 3	Implement scaled dot-product attention, the causal mask and multi-head attention, and identify a specialised attention head in a real model (K1).
Exercise 4	Implement and compare decoding strategies, and measure their effect on quality, variety and speed (K1).
Exercise 5	Compare a base and an instruction-tuned model, and adapt a small model to an Arabic task with LoRA (K2).
Exercise 6	Build a prompt test set and an automatic scorer, and use them to compare prompt designs (S1).

Exercise 1: Hello, LLM

Week 1 · Notebook: labs/lab-01-hello-llm/Lab_01_Hello_LLM.ipynb · Graded by: assignment (notebook and failure log), 5 marks

Exercise Objectives:

- Run an open-weight LLM on a lab GPU or on Google Colab.
- Read a model's real next-token probabilities, in English and Arabic.
- Write the generation loop and see that every token costs one forward pass.
- Trigger the six failure types from the lecture and explain each from how the model works.

Background:

A language model computes P(next token | all previous tokens): one probability for every token in its vocabulary. Generation repeats one step: run the model, pick a token, append it and run again. Picking the most likely token each time is greedy decoding. Instruction-tuned models read a conversation turned into one token sequence by a chat template. Six failures follow directly from this design: hallucination, stale knowledge, letter counting, prompt sensitivity, prompt injection and uneven quality across languages.

In class Experiments (for testing):

- Part A: show the top-5 next tokens for given prefixes in English and Arabic, then for three prefixes of your own.
- Part B: complete generate_greedy (two TODO lines) and compare its output and speed with model.generate.
- Part C: print the chat template; ask one question in English, MSA and Jordanian Arabic; compare answers at temperature 0 and 1.2.

Experiment #1 (for grading):

Assignment (5 marks), submitted on Moodle: the completed notebook, run from top to bottom with outputs, and lab01_failures.csv.

- Parts A–C completed and answered (1.5 marks).
- Part D failure log: at least 12 logged attempts covering all six failure types in both languages, each explained (2 marks).
- Part E reflection: three answers that explain the results from how the model works (1.5 marks).

Results / Observations:

- The probability tables for your prefixes, and which one was flattest.
- The output and timing of your loop against model.generate.
- The failure log and its coverage table (six failure types × two languages).

Discussion / Conclusion:

- In one sentence, what does a language model compute?
- Which failure was easiest to trigger in Arabic, and which in English? Explain the difference from how the model works.
- Did a firm system message stop your prompt-injection attempt? Why is this hard to fix completely?

Exercise 2: Tokens and Embeddings

Week 2 · Notebook: labs/lab-02-tokens-embeddings/Lab_02_Tokens_and_Embeddings.ipynb · Graded by: assignment (tokenizer audit), 5 marks

Exercise Objectives:

- Tokenize the same text with five tokenizers and explain the differences.
- Measure fertility (tokens per word) and the Arabic ÷ English token ratio, and turn them into cost.
- Compare static and contextual embeddings, and match sentences across languages.

Background:

Models read tokens, not words. A tokenizer's vocabulary is learned from its training text (BPE, WordPiece or SentencePiece), so text in a language that was rare in that text is split into many small pieces, which raises cost and shrinks the usable context. Each token then becomes a row of a learned embedding table; the layers above turn these static vectors into contextual ones that depend on the whole sentence.

In class Experiments (for testing):

- Part A: tokenize one sentence pair with GPT-2, Qwen2.5, mBERT, XLM-R and AraBERT.
- Part B: measure fertility and the Arabic ÷ English ratio on 20 sentence pairs and 5 dialect sentences; plot the result.
- Part C: compare static and contextual similarity for ambiguous words (bank, العين), then match 20 MSA and 5 dialect sentences to their English translations.

Experiment #2 (for grading):

Assignment (5 marks): the tokenizer audit, individual, due at the start of the week 3 lab (brief: labs/lab-02-tokens-embeddings/MA1_Tokenizer_Audit.md).

- Add at least one more tokenizer and report fertility and the token ratio for all of them, in one table and one chart.
- Write 10 dialect sentences of your own with MSA and English versions, and test whether the dialect gap holds.
- Report cross-lingual matching accuracy for MSA and dialect, and explain two misses.

Four criteria are scored 0, 1 or 2; the total (out of 8) is scaled to 5 marks.

Results / Observations:

- The token counts per tokenizer, the fertility table and the ratio chart.
- Static and contextual cosine similarities, and the top-1 matching accuracy for MSA and dialect.

Discussion / Conclusion:

- Why can the same sentence need three times as many tokens with one tokenizer as with another?
- What does a high Arabic ÷ English ratio cost a real application?
- Why does the embedding table give "bank" the same vector in every sentence, and what fixes this?

Exercise 3: Attention from Scratch

Week 3 · Notebook: labs/lab-03-attention/Lab_03_Attention_from_Scratch.ipynb · Graded by: assignment (notebook), 5 marks

Exercise Objectives:

- Implement scaled dot-product attention and check it against PyTorch.
- Add a causal mask and explain why a decoder needs it.
- Build a multi-head attention layer.
- Find an attention head with a clear job inside Qwen2.5, in English and in Arabic.

Background:

Self-attention lets every token gather information from the others: its query is compared with every key, the scores are scaled by √d and turned into weights with softmax, and the output is the weighted sum of the values. A causal mask stops a token from seeing the future. Several heads run in parallel, each free to learn a different relation, such as attending to the previous token.

In class Experiments (for testing):

- Part A: attention for one query by hand, then for all queries at once, checked against PyTorch.
- Part B: show that attention without positions cannot see word order.
- Part C: see why the scores are divided by √d, by watching softmax saturate.
- Part D: the causal mask (D1) and a multi-head attention layer (D2), each checked against PyTorch.

Experiment #3 (for grading):

Assignment (5 marks): the completed notebook with outputs, submitted on Moodle.

- Exercises A, D1 and D2: attention, the causal mask and the multi-head layer pass their checks (3 marks).
- Part E: the previous-token head found in Qwen2.5 and tested in Arabic, and where "it" looks (1 mark).
- Part F: written answers to the reflection questions (1 mark).

Results / Observations:

- The check outputs of Parts A–C, the attention heatmaps and the head scores.

Discussion / Conclusion:

- In one sentence each, what do a token's query, key and value do?
- Why are the scores divided by √d?
- In "…because it was too tired / wide", why does "it" attend the same way in both sentences?

Exercise 4: Decoding

Week 4 · Notebook: labs/lab-04-decoding · Graded by: assignment (notebook), 5 marks

Exercise Objectives:

- Implement greedy, temperature, top-k and top-p sampling, and use beam search.
- Measure how each strategy changes quality, variety and repetition.
- Measure the speed-up a KV cache gives during generation.

Background:

The model outputs a probability distribution; the decoding strategy decides which token is chosen. Greedy decoding and beam search favour likely text but repeat themselves; temperature reshapes the distribution; top-k and top-p cut off its long tail. Because each step re-reads the whole prefix, generation caches the keys and values of earlier tokens (the KV cache).

In class Experiments (for testing):

- Implement the samplers on the logits of Qwen2.5 and generate continuations for English and Arabic prompts.
- Compute distinct-n and a repetition rate for each strategy over several samples.
- Time generation with and without the KV cache.

Experiment #4 (for grading):

Assignment (5 marks): the completed notebook with outputs, submitted on Moodle.

- Samplers implemented and checked (2 marks).
- Comparison table of strategies with the variety and repetition measures (2 marks).
- Short analysis: which settings you would use for a factual assistant and for creative writing, and why (1 mark).

Results / Observations:

- Sample outputs per strategy, the comparison table and the KV-cache timing.

Discussion / Conclusion:

- What do we gain and lose by always picking the most likely token?
- How do temperature and top-p interact?

Exercise 5: Adapting a Model

Week 5 · Notebook: labs/lab-05-adapting · Graded by: assignment (fine-tuning report), 5 marks

Exercise Objectives:

- Compare a base model with its instruction-tuned version on the same prompts.
- Fine-tune a small open-weight model on an Arabic task with LoRA.
- Evaluate the model before and after fine-tuning on held-out examples.

Background:

Pre-training gives a model its knowledge; instruction tuning and preference alignment shape its behaviour. Full fine-tuning updates every weight, while LoRA trains two small low-rank matrices beside each weight matrix, so a model can be adapted on a single GPU. Choosing between prompting, retrieval and fine-tuning is an engineering decision.

In class Experiments (for testing):

- Run the same prompts through a base and an instruction-tuned Qwen2.5 model and compare the answers.
- Fine-tune a small model with LoRA on a provided Arabic dataset; watch the training loss.
- Count trainable parameters with and without LoRA.

Experiment #5 (for grading):

Assignment (5 marks): the notebook and a one-page report, submitted on Moodle.

- Fine-tuning run completed, with the loss curve (2 marks).
- Before/after evaluation on the held-out set (2 marks).
- Analysis: when would you fine-tune instead of prompting or retrieving (1 mark).

Results / Observations:

- Base and instruct outputs side by side, the loss curve, the trainable-parameter counts and the before/after scores.

Discussion / Conclusion:

- Why can LoRA adapt a model on one GPU when full fine-tuning cannot?
- What did fine-tuning improve, and what did it not?

Exercise 6: A Prompt Test Harness

Week 6 · Notebook: labs/lab-06-prompt-harness · Graded by: assignment (notebook), 5 marks

Exercise Objectives:

- Write a small test set with expected answers in English and Arabic.
- Build an automatic scorer and run prompts against the test set.
- Compare zero-shot, few-shot and format-constrained prompts with numbers rather than by eye.

Background:

A prompt is the whole input: system message, instructions, examples and the user's text. Small changes can change the answer, so prompt engineering is measured like any other engineering change: a fixed test set, an automatic score, and one change at a time.

In class Experiments (for testing):

- Write at least 20 test cases with expected answers, at least half in Arabic.
- Implement an exact-match or rule-based scorer and a harness that runs a prompt over the whole set.
- Score a zero-shot prompt, a few-shot prompt and a prompt with a strict output format.

Experiment #6 (for grading):

Assignment (5 marks): the completed notebook with outputs, submitted on Moodle.

- Test set of 20+ cases with expected answers (1 mark).
- Working scorer and harness (2 marks).
- Comparison of three prompt designs with a short analysis of the failures (2 marks).

Results / Observations:

- The test set, the score of each prompt design and the cases that still fail.

Discussion / Conclusion:

- Which change helped most, and on which kind of question?
- How large must a test set be before a difference in score means something?